Compiler Project Work Plan

Nov 6, 2026 · @ansa

The work splits into three owned areas with fixed interfaces, so all three people can start on day one and merge without blocking each other. The language itself is defined in the revised proposal; when this plan and the proposal disagree, the proposal wins.

Roles

Each person owns one area end to end, in both phases. Names are a starting suggestion; swap roles freely, but keep one owner per file.

Area	Owner	Phase 1 (interpreter)	Phase 2 (LLVM IR)	Files owned
Front end and integration	Hareem	Lexer, parser, AST, AST printer, driver, Makefile, test runner	Constant folding, unreachable-code removal, middle-end hookup, README.pdf	`src/lexer.l`, `src/parser.y`, `src/ast.h`, `src/ast.cpp`, `src/main.cpp`, `src/opt.cpp`, `Makefile`, `tests/run.sh`
Semantics and interpreter	Aleena	Symbol table, type checker, every compile-time check, tree-walking interpreter	IR for recipes, calls, `give`, rows (create, index, bounds check, `count`)	`src/sema.h`, `src/sema.cpp`, `src/interp.cpp`, `src/codegen_recipes.cpp`
Runtime and core codegen	Alizeh	C runtime library: page loading, selectors, `grab`, number conversion, `save`, `show`, run-time errors	Codegen context, IR for expressions, variables, `if` / `while` / `for`, short-circuit `and` / `or`, runtime calls, `@main`	`runtime/rt.h`, `runtime/rt.c`, `src/codegen.h`, `src/codegen.cpp`

The split is uneven by phase on purpose: Hareem's front end is the first thing everyone needs, so it is front-loaded; Hareem then takes optimizations, testing and the report while the other two carry Phase 2 codegen.

Contracts to agree on day one

These five things are fixed together in the first meeting, before anyone writes feature code. After that, a change to any of them needs a message to the group.

1. Repo layout and tools. C++17 for the compiler, C for the runtime, Flex and Bison with the plain C interface, textual LLVM IR (no LLVM C++ API, so nobody needs LLVM dev headers). One Git repo; everyone works on a branch and merges through a pull request

that one other member reviews.

```
src/        lexer.l  parser.y  ast.h  ast.cpp  main.cpp  opt.cpp
            sema.h  sema.cpp  interp.cpp  codegen.h  codegen.cpp
codegen_recipes.cpp
runtime/    rt.h  rt.c
tests/ok/   *.dsl + matching *.out (expected stdout)
tests/err/  *.dsl, first line: # expect: <line>: <part of the message>
examples/   isort.dsl  msort.dsl  prices.dsl  headlines.dsl  shop.html
news.html
Makefile    README
```

2. AST (owner: Hareem, reviewed by all). `ast.h` lists every node kind from the grammar in the proposal. Every node stores its source `line`. Expression nodes have a `type` field (`NUM`, `TEXT`, `ROW_NUM`, `ROW_TEXT`, `VOID`) that semantic analysis fills in. Variable and call nodes have a pointer to their symbol, also filled in by semantic analysis, so the interpreter and codegen never repeat name lookup.

3. Runtime API (owner: Alizeh). The interpreter calls these directly; the generated IR declares and calls the same functions. In IR a row is `%Row = type { i64, i64, ptr }`, and `count(r)` is a load of field 0.

```
typedef struct { int64_t len; int64_t is_text; void *data; } Row;  /* data:
int64_t* or char** */

void    rt_open(const char *path, int64_t line);
Row    *rt_grab(const char *selector, int64_t as_num, int64_t line);
Row    *rt_row_new(int64_t n, int64_t is_text, int64_t line);
void    rt_check_index(Row *r, int64_t i, int64_t line);
void    rt_check_div(int64_t divisor, int64_t line);
int64_t rt_text_eq(const char *a, const char *b);
void    rt_show_num(int64_t v);
void    rt_show_text(const char *s);
void    rt_show_row(Row *r);
void    rt_save_num(int64_t v, const char *path, int64_t line);
void    rt_save_text(const char *s, const char *path, int64_t line);
void    rt_save_row(Row *r, const char *path, int64_t line);
int     rt_selector_valid(const char *sel);   /* also called by semantic
analysis */
```

4. Error format. Compile-time: `prog.dsl:12: error: <message>` on stderr, exit code 1. Run-time: `line 12: runtime error: <message>` on stderr, exit code 1. Tests match on the line number and part of the message, so wording can change.

5. Command line.

Command	Does	Needed by
`./dslc prog.dsl --ast`	Parse and print the AST	Milestone 1
`./dslc prog.dsl --check`	Parse and run semantic analysis only	Milestone 2
`./dslc prog.dsl --run`	Interpret the program	Milestone 2 (Phase 1)
`./dslc prog.dsl -o prog.ll`	Emit LLVM IR	Milestone 3 (Phase 2)

The IR is then passed through the provided middle-end and linked: `clang prog.ll runtime/rt.c $(xml2-config --cflags --libs) -o prog`.

Milestones

Six checkpoints, in order; a milestone is reached only when its "Done when" check passes on a clean checkout of `main`. Fill in dates once the Phase 1 and Phase 2 deadlines are confirmed.

Milestone	Hareem	Aleena	Alizeh	Done when
M0 Setup	Repo, Makefile skeleton, draft `ast.h`	Write the 4 example programs and their expected output	Draft `rt.h`, write `shop.html` and `news.html` test pages	Contracts merged; each person has added 5 error programs to `tests/err/`

Milestone	Hareem	Aleena	Alizeh	Done when
M1 Front end	Lexer, parser, AST, `--ast` printer	Symbol table and type rules, written against `ast.h`	Runtime library, tested on its own with a small C test program	`--ast` prints a tree for every example; the C test prints the prices grabbed from `shop.html`
M2 Phase 1 done	Driver, `--check` / `--run`, test runner, syntax-error tests	All semantic checks, interpreter	Hook runtime into the interpreter, run-time error tests	Every `tests/ok` program gives its expected output under `--run`; every `tests/err` program gives its expected error
M3 Core IR	Constant folding, unreachable-code removal; test runner also compiles and runs binaries	IR for recipes, calls, `give`, rows	Codegen context; IR for expressions, variables, control flow, runtime calls	Every `tests/ok` program without recipes compiles, and its binary prints the same output as `--run`
M4 Phase 2 done	Hook up the provided middle-end	Recursion and row tests (Merge Sort, in-place sorting)	Check all emitted IR with `llvm-as` and `opt -verify`	Every `tests/ok` program gives identical output in both modes
M5 Submission	README.pdf: build steps, usage, language summary, test results	README section: type rules and errors	README section: runtime and IR mapping	Full build and test run on a fresh Linux machine following only the README

Two dependencies to watch: semantic analysis cannot be tested until the parser builds real ASTs (end of M1), and Phase 2 codegen needs the typed AST from M2. Alizeh can start the codegen context during M2 once the runtime is done.

Task checklists

Tick items off as they merge into `main`. Each list is in the order the work should happen.

Hareem — front end and integration

- [ ] Lexer: every keyword, identifiers, number literals (error if too large for 64 bits), text literals with `\"`, `\\`, `\n` escapes, `#` comments
- [ ] Lexer: longest match for operators that share a first character (`<-` `<<` `<=` `<`, `->` `-`, `>>` `>=` `>`); a lone `=` is an error
- [ ] Lexer: one `NL` token per line that has a token, plus one at end of file if the last line has no newline; track line numbers
- [ ] Parser: full grammar from the proposal, precedence with `%left` / `%nonassoc` (comparisons non-associative), zero shift/reduce conflicts (check with `bison -Wcounterexamples`)
- [ ] Parser: `<< else >>` on one line, `else if` chains, syntax errors that name the line and the unexpected token
- [ ] AST construction and the `--ast` printer
- [ ] Driver (`main.cpp`) with all four flags; Makefile targets `make`, `make test`, `make clean`
- [ ] `tests/run.sh`: runs `tests/ok` and `tests/err` under `--run`; in Phase 2 also compiles each `tests/ok` program and compares the binary's output with `--run`
- [ ] `opt.cpp`: constant folding of num expressions with literal operands (never fold `/ 0` or `% 0`); remove statements after a `give` in the same block
- [ ] Pass the IR through the provided middle-end inside the build and test scripts
- [ ] README.pdf

Aleena — semantics and interpreter

- [ ] Symbol table: a stack of scopes for variables, a separate table for recipes, `count` reserved
- [ ] Pass 1: collect every recipe signature; report duplicate recipes
- [ ] Pass 2: declarations, assignments and the type of every expression, following the type and operator rules in the proposal
- [ ] Recipe checks: argument count and types, a no-return recipe used as a value, all four `give` rules

- [ ] Assigning to a `for` loop variable is an error
- [ ] Domain checks: selector validity (call `rt_selector_valid`), no `open` / `grab` in recipes, every `grab` preceded by an `open` in the same or an enclosing block (a flag per scope that inner blocks inherit and outer blocks do not)
- [ ] Keep going after an error and report all of them; give a bad expression an `ERROR` type so one mistake does not cause a cascade
- [ ] Interpreter: values (num, text, row pointer), a fresh variable frame per call, short-circuit `and` / `or`, `for` bounds evaluated once, `give` as a return signal, default return values, all run-time checks through the `rt_*` functions
- [ ] IR: one `define` per recipe with typed parameters (params copied into allocas), calls, `give` as `ret`, a default `ret` at the end of the body
- [ ] IR: rows — `rt_row_new`, `rt_check_index` then `getelementptr` into `data` for element load and store, `count` as a load of field 0

Alizeh — runtime and core codegen

- [ ] `rt_open`: parse with libxml2 `htmlReadFile` (recover mode, warnings off); keep one current page and free the previous one
- [ ] Selector parser for the supported subset, turned into an XPath query (descendants joined with `//`, a class matched with `contains(concat(' ', normalize-space(@class), ' '), ' price ')`); `rt_selector_valid` uses the same parser
- [ ] `rt_grab`: text content of each match, trimmed with inner whitespace collapsed; number conversion exactly as the proposal states, with its run-time error
- [ ] Rows, `rt_show_*`, `rt_save_*`, `rt_check_index`, `rt_check_div`, `rt_text_eq`; every error prints the source line and exits with 1
- [ ] `runtime/test_rt.c`: a standalone test that opens both test pages and prints what it grabs
- [ ] Codegen context: `emit()`, fresh temporaries and labels, a pool of string constants (`@.str.N`), all allocas placed in the entry block, `declare` lines for every runtime function
- [ ] Expressions: i64 arithmetic, `sdiv` / `srem` after `rt_check_div`, comparisons as `icmp` + `zext` to i64, text `==` / `!=` through `rt_text_eq`, unary minus and `not`
- [ ] Conditions as `icmp ne 0`; short-circuit `and` / `or` with branches and a `phi`
- [ ] `if` / `else`, `while`, `for` (bounds evaluated once) as basic blocks; every block ends in exactly one terminator, including after a `give`

- [ ] `open`, `grab`, `save`, `show` as runtime calls; `@main` returns 0

Testing and definition of done

The interpreter is the reference for Phase 2: a compiled program is correct when its output matches `--run` on the same program.

- **Correct programs** (`tests/ok/`): the 4 examples plus small programs for each feature — scopes and shadowing, `else if`, `for` with an empty range, text equality, `grab` with zero matches, a row passed to a recipe and changed there, recursion. Each has a `.out` file with the expected stdout.
- **Error programs** (`tests/err/`): at least one program per error in the proposal's error list, compile-time and run-time. The first line states the expected line number and part of the message.
- **Who writes them:** each person writes the tests for their own area, and one test per error category is written by someone other than its owner, to catch misreadings of the proposal.

A task is done when it is merged into `main`, has tests in `tests/`, `make test` passes, and one other person has reviewed it.

Risks to check in the first week

Risk	Check	Owner
The grading machine lacks libxml2	Confirm `libxml2-dev` installs on the lab image; put the install command in the README	Alizeh
The provided middle-end expects a particular IR version or entry point	Run one hand-written `.ll` file through it before M3	Hareem
Grammar conflicts around `NL`, `else` and calls vs assignment	Get the full grammar conflict-free in M1, before semantic work depends on it	Hareem
Proposal and code drift apart	Any change to the language goes into the proposal first, then the code	All

Risk	Check	Owner
Invalid IR that only fails at link time	Run `llvm-as` on every emitted `.ll` inside `make test`	Alizeh